Programmieraufgabe: Fuhrpark-Verwaltung

Für: Michael **Thema:** Vererbung, Polymorphie, Abstraktion und Interfaces in einem zusammenhängenden Projekt **Art:** Konsolenprojekt in C# (Visual Studio, Projekttyp *Console App*) **Bearbeitungszeit:** ca. 3–4 Stunden, verteilt auf 2–3 Sessions **Voraussetzung:** Du hast die vier Einzelaufgaben (Vererbung, Polymorphie, Abstraktion, Interfaces) gemacht. Hier kommt alles zusammen.

---

1. Worum geht es?

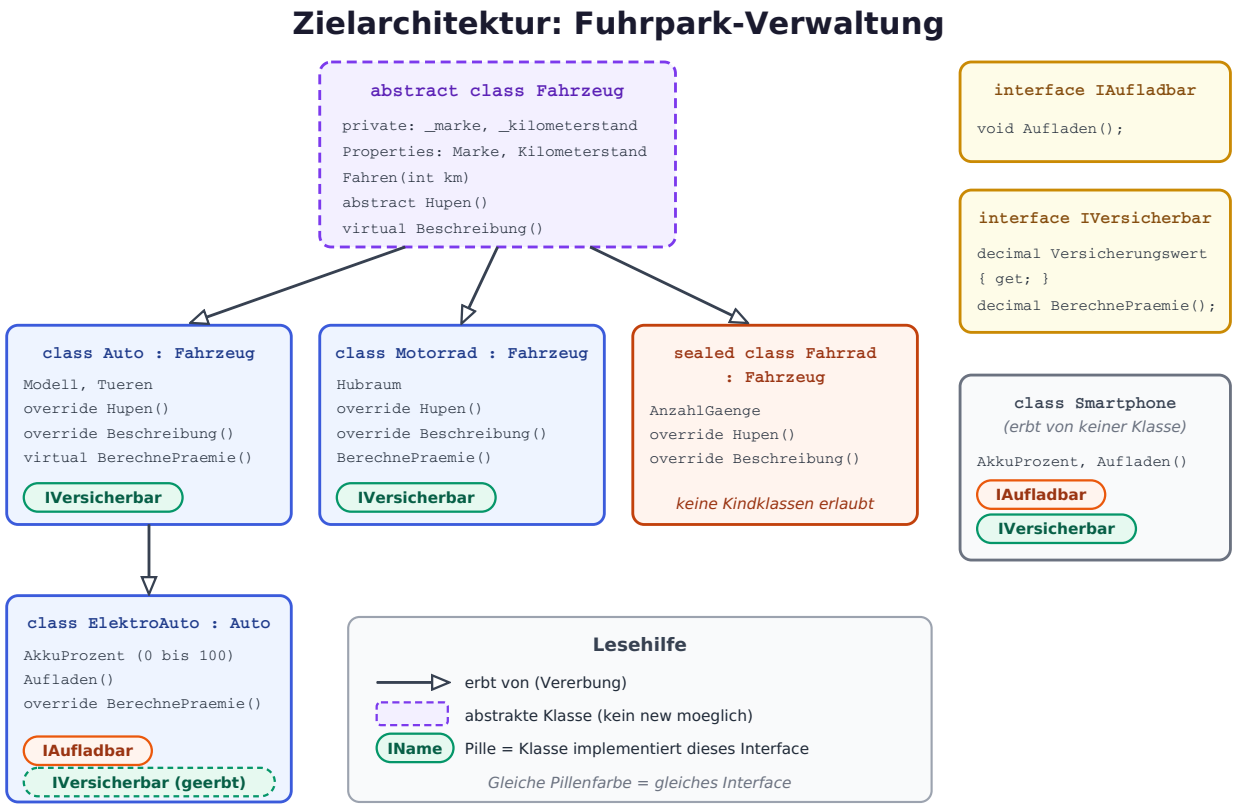
Du baust eine kleine Verwaltung für einen Fuhrpark. Fahrzeuge verschiedener Art werden erfasst, angezeigt, bewegt, aufgeladen und versichert. Dabei brauchst du **jedes Konzept** aus den bisherigen Aufgaben mindestens einmal, und zwar nicht zum Selbstzweck, sondern weil das Programm es sinnvoll verlangt.

Die Aufgabe gibt dir **keine Lösung** vor, nur den Vertrag: Welche Klassen gibt es, was müssen sie können, welche Regeln gelten. Wie du es umsetzt, entscheidest du.

2. Welche Themen kommen vor?

Thema	Wo es in dieser Aufgabe vorkommt	Nachlesen
Klassen, Objekte, Konstruktoren	alle Klassen	Constructors
Access Modifiers, Properties	private Felder, Property-Validierung	Properties
Vererbung, `base(...)`, `sealed`	`Fahrzeug` und alle Kindklassen	Inheritance
`virtual` / `override`, Polymorphie	`Beschreibung()`, `BerechnePraemie()`, Listen	Polymorphism
Abstrakte Klasse und Methode	`Fahrzeug`, `Hupen()`	Abstraction
Interface, mehrere Interfaces	`IAufladbar`, `IVersicherbar`, `Smartphone`	Interface, Multiple Interfaces
Listen, `foreach`, `switch`, Schleifen	Menü, Auswertungen	Foreach, Switch

3. Das Zielbild



Das Bild zeigt **was** es gibt und **wie die Klassen zusammenhängen**, nicht wie der Code aussieht. Lies es genau: Welche Klasse ist abstrakt? Wo steht `sealed`? Welche Klasse erbt von keiner anderen? Welche Klasse bekommt ein Interface “geerbt”, ohne es selbst hinzuschreiben?

---

4. Der Vertrag (Anforderungen)

4.1 Klassen und Interfaces

Typ	Erbt von / implementiert	Pflicht-Inhalt
`Fahrzeug`	abstrakt, keine Basisklasse	Marke und Kilometerstand als **private Felder** mit **Properties**; Konstruktor mit Marke; Methode `Fahren(int km)`; abstrakte Methode `Hupen()`; virtuelle Methode `Beschreibung()`
`Auto`	`Fahrzeug`, `IVersicherbar`	Modell, Türen; eigenes `Hupen()` und `Beschreibung()`; Prämienberechnung, die in Kindklassen angepasst werden darf
`Motorrad`	`Fahrzeug`, `IVersicherbar`	Hubraum; eigenes `Hupen()` und `Beschreibung()`; eigene Prämienberechnung
`Fahrrad`	`Fahrzeug`, **versiegelt**	Anzahl Gänge; eigenes `Hupen()` und `Beschreibung()`; **nicht** versicherbar
`ElektroAuto`	`Auto`, `IAufladbar`	Akkustand in Prozent; `Aufladen()`; angepasste Prämienberechnung
`Smartphone`	keine Basisklasse, `IAufladbar`, `IVersicherbar`	Akkustand in Prozent; `Aufladen()`; Versicherungswert; Prämienberechnung
`IAufladbar`	Interface	Methode `Aufladen()`
`IVersicherbar`	Interface	Property `Versicherungswert` (nur lesbar), Methode `BerechnePraemie()` mit Rückgabetyp `decimal`

4.2 Regeln, die dein Code einhalten muss

Fahrzeug

- `Marke` wird nur im Konstruktor gesetzt und ist danach nur lesbar.
- Der Kilometerstand darf **nie sinken**. `Fahren(int km)` akzeptiert nur Werte grösser als 0 und erhöht den Stand entsprechend. Bei ungültigem Wert passiert nichts am Stand.
- `Beschreibung()` liefert in der Basisklasse Marke und Kilometerstand als Text. Jede Kindklasse **ergänzt** ihre eigenen Angaben, ohne den Basistext zu kopieren.
- `Hupen()` gibt je Klasse einen eigenen Text aus (z.B. `Hup, hup!`, `Brumm!`, `Kling, kling!`).

Akkustand (bei `ElektroAuto` und `Smartphone`)

- Property mit Wertebereich 0 bis 100. Ungültige Werte werden **nicht** gespeichert.
- `Aufladen()` setzt den Akku auf 100 und gibt eine Meldung aus.

Versicherung (`decimal` verwenden, es geht um Geld)

- Versicherungswert wird per Konstruktor übergeben.
- Prämie als Anteil des Versicherungswerts: Auto 3 %, Motorrad 5 %, Smartphone 8 %.
- `ElektroAuto`: 10 % Rabatt auf die normale Auto-Prämie. Die Berechnung der Auto-Prämie darfst du **nicht** kopieren, sondern musst sie wiederverwenden.
- Tipp zur Wiederverwendung: Im Konstruktor hast du mit `base(...)` schon etwas Ähnliches kennengelernt. Gibt es das auch für Methoden?

4.3 Das Programm (Menü)

Das Programm läuft in einer Schleife, bis der Nutzer beendet. Menü mit `switch`:

Nr.	Funktion
1	Auto hinzufügen
2	Motorrad hinzufügen
3	Fahrrad hinzufügen
4	ElektroAuto hinzufügen
5	Smartphone hinzufügen
6	Alle Fahrzeuge anzeigen (`Beschreibung()`)
7	Alle Fahrzeuge hupen lassen
8	Fahrt erfassen (Fahrzeug aus der Liste wählen, Kilometer eingeben)
9	Alle aufladbaren Geräte aufladen
10	Versicherungsübersicht: Prämie pro Objekt und Gesamtsumme
0	Beenden

Drei Listen halten die Objekte, je nach Fähigkeit:

- `List<Fahrzeug>`: alle Fahrzeuge (Auto, Motorrad, Fahrrad, ElektroAuto)
- `List<IAufladbar>`: alle aufladbaren Objekte (ElektroAuto, Smartphone)
- `List<IVersicherbar>`: alle versicherbaren Objekte (Auto, Motorrad, ElektroAuto, Smartphone)

Ein neu erfasstes Objekt kommt in **alle** Listen, zu denen es passt. Ein `ElektroAuto` landet also in drei Listen, ein `Fahrrad` in einer.

Eingaben prüfen: Leere Marke ablehnen, Zahlen mit `TryParse` einlesen, ungültige Menüauswahl abfangen. Das Programm darf bei Falscheingaben nicht abstürzen.

4.4 Harte Regeln (Qualität)

- **Die Schleifen in den Menüpunkten 6, 7, 9 und 10 dürfen den konkreten Typ nicht kennen.** Sie arbeiten nur mit `Fahrzeug`, `IAufladbar` bzw. `IVersicherbar`.
- Keine Kopien von Code zwischen Geschwisterklassen. Was alle Fahrzeuge gemeinsam haben, steht in `Fahrzeug`.
- Felder sind `private`, Zugriff läuft über Properties oder Methoden.
- Aussagekräftige Namen, jede Klasse in einer eigenen Datei.

---

5. Vorgehen in Etappen

Arbeite die Etappen **der Reihe nach** ab und teste nach jeder Etappe. Nach jeder Etappe läuft das Programm, es wächst nur.

Etappe 0 – Planen, bevor du tippst (ca. 20 Min.)

1. Zeichne auf Papier die Klassenhierarchie aus dem Zielbild **aus dem Kopf nach**, ohne hinzusehen. Was fehlt dir noch?
2. Zeichne ein zweites Bild: Wo liegen später die Objekte im Speicher, wenn ein `ElektroAuto` in drei Listen steht? (Stack, Heap, Pfeile.)
3. Lege das Projekt an und erstelle für jede Klasse und jedes Interface eine leere Datei.

Etappe 1 – Die abstrakte Basis (ca. 30 Min.)

- `Fahrzeug` mit privaten Feldern, Properties, Konstruktor, `Fahren(int km)`, abstraktem `Hupen()` und virtuellem `Beschreibung()`.
- `Auto` und `Motorrad` als erste Kindklassen mit allem, was sie als Pflichtinhalt brauchen

(Versicherung kommt erst in Etappe 4).
- Teste in `Main` mit festen Objekten, ohne Menü.

Vorhersage zuerst: Du rufst `Fahren(-50)` auf einem frischen Fahrzeug auf. Was steht danach im Kilometerstand? Notiere deine Vermutung, dann prüfen.

Etappe 2 – Polymorphie (ca. 30 Min.)

- Ergänze `Fahrrad` (noch ohne `sealed`).
- Lege eine `List<Fahrzeug>` mit einem Auto, einem Motorrad und einem Fahrrad an. Lass in einer `foreach`-Schleife jedes Fahrzeug hupen und seine Beschreibung ausgeben.
- **Experiment:** Entferne vorübergehend `virtual` bei `Beschreibung()` und `override` in einer Kindklasse. Sage vorher voraus, was die Ausgabe ändert. Dann zurückbauen.
- **Debugger-Auftrag:** Setze einen Breakpoint in der `foreach`-Schleife und gehe mit F11 in `Hupen()` hinein. Beobachte bei jedem Durchlauf, in welcher Klasse du landest.

Etappe 3 – Vererbungskette und `sealed` (ca. 20 Min.)

- Mache `Fahrrad` versiegelt.
- Ergänze `ElektroAuto`, das von `Auto` erbt, mit Akkustand und `Aufladen()`.
- Prüfe, dass `ElektroAuto` alles hat, was `Auto` hat, ohne dass du es erneut geschrieben hast.

Etappe 4 – Interfaces (ca. 40 Min.)

- Definiere `IAufladbar` und `IVersicherbar`.
- `Auto`, `Motorrad` und `ElektroAuto` werden versicherbar, `ElektroAuto` zusätzlich aufladbar.
- Baue `Smartphone` als Klasse **ohne** Basisklasse, die beide Interfaces implementiert.
- Prämienberechnung: Auto virtuell, `ElektroAuto` überschreibt mit Rabatt und verwendet dabei die Berechnung von `Auto` weiter.
- Teste mit `List<IAufladbar>` und `List<IVersicherbar>` und je einer `foreach`-Schleife.

Etappe 5 – Das Menü (ca. 40 Min.)

- Baue das Menü aus 4.3 mit `switch` in einer Schleife.
- Alle drei Listen werden beim Hinzufügen befüllt.
- Eingabeprüfung wie in 4.3 beschrieben.

Etappe 6 – Fehler-Labor (ca. 20 Min.)

Baue nacheinander die folgenden Fehler absichtlich ein, lies die Meldung und trage ein, was du vorhergesagt hast, was der Compiler wirklich sagt (in **eigenen Worten**) und was du daraus lernst. Danach jeweils zurückbauen.

#	Experiment	Meine Vorhersage	Meldung in eigenen Worten	Was habe ich gelernt?
1	`new Fahrzeug("X");` erzeugen			
2	In einer Kindklasse `Hupen()` nicht überschreiben			
3	Eine Klasse von `Fahrrad` erben lassen			
4	Im Interface ein Feld deklarieren			
5	Interface-Methode in `Smartphone` ohne `public` schreiben			
6	`override` schreiben, aber `virtual` in der Basis weglassen			
7	`IAufladbar x = new IAufladbar();`			

Etappe 7 – Erweiterungstest (ca. 20 Min.)

Ergänze **eine neue Klasse** deiner Wahl, z.B. `Lastwagen : Fahrzeug` (versicherbar) oder `Tablet` (aufladbar und versicherbar), und ein Menüpunkt zum Hinzufügen.

Regel: In den Auswertungsschleifen (Menüpunkte 6, 7, 9, 10) darfst du **keine einzige Zeile** ändern. Wenn du es doch musst, ist irgendwo ein Typ zu konkret geworden. Finde die Stelle und behebe die Ursache.

---

6. Abnahme-Tests (so prüfst du dich selbst)

Das Programm gilt als fertig, wenn alle diese Beobachtungen stimmen:

- ☐ Nach dem Hinzufügen von Auto, Motorrad und Fahrrad gibt Menüpunkt 7 **drei verschiedene** Hupen-Texte aus, ohne dass die Schleife die Typen kennt.
- ☐ Menüpunkt 6 zeigt für jedes Fahrzeug eine Beschreibung, die Marke und Kilometerstand **und** die klassenspezifischen Angaben enthält.
- ☐ `Fahren(0)` und `Fahren(-5)` ändern den Kilometerstand nicht.
- ☐ Ein `Fahrrad` taucht in Menüpunkt 10 **nicht** auf, ein `Smartphone` schon.
- ☐ Die Prämie eines `ElektroAuto` ist niedriger als die eines `Auto` mit gleichem Versicherungswert.
- ☐ Ein `Smartphone` taucht in Menüpunkt 9 und 10 auf, aber **nicht** in Menüpunkt 6 und 7.
- ☐ Ungültige Eingaben (Buchstaben statt Zahl, leere Marke, Akku 150) bringen das Programm nicht zum Absturz.
- ☐ `new Fahrzeug(...)` und `new IAufladbar()` lassen sich nicht kompilieren. Eine Klasse, die von `Fahrrad` erbt, ebenfalls nicht.
- ☐ Die Erweiterung aus Etappe 7 funktioniert, ohne dass sich die Auswertungsschleifen ändern.

---

7. Erklär-Check (mündlich, mit deinen Zeichnungen)

Beantworte diese Fragen **ohne** in den Code zu schauen. Zeichne, wenn es hilft.

1. Warum ist `Fahrzeug` abstrakt und nicht einfach eine normale Klasse? Was wäre im Programm anders, wenn man `new Fahrzeug(...)` erlauben würde?
2. `Hupen()` ist abstrakt, `Beschreibung()` ist virtuell. Warum hast du für die beiden Methoden unterschiedliche Schlüsselwörter gewählt?
3. Wie weiss die `foreach`-Schleife in Menüpunkt 7, welchen Text sie ausgeben muss, obwohl sie nur `Fahrzeug` kennt?
4. Warum ist `Smartphone` kein Kind von `Fahrzeug`, kann aber trotzdem in einer Liste mit `ElektroAuto` stehen? Was verbindet die beiden?
5. `ElektroAuto` erbt von `Auto` und implementiert zusätzlich ein Interface. Wie viele Basisklassen und wie viele Interfaces hat es insgesamt? Woher kommt `IVersicherbar`?
6. Wofür hast du `sealed` bei `Fahrrad` eingesetzt? In welchem Fall würdest du es nicht einsetzen?
7. Ein `ElektroAuto` steht in drei Listen. Du erfasst über die `List<Fahrzeug>` eine Fahrt. Siehst du die neuen Kilometer auch, wenn du es über die `List<IVersicherbar>` anschaust? Erkläre es mit deinem Speicherbild.
8. Welche Stelle in deinem Programm würdest du als Erstes ändern müssen, wenn eine neue Fahrzeugart dazukommt? Und welche Stelle zuletzt?

---

8. Bewertungsraster (für die Besprechung mit Jasmin)

Pro Zeile: erfüllt, teilweise oder offen.

Bereich	Kriterium	Stand
Abstraktion	`Fahrzeug` abstrakt, `Hupen()` abstrakt, `Beschreibung()` virtuell	
Vererbung	Kindklassen nutzen `base(...)` im Konstruktor, keine duplizierten Felder	
Vererbung	Mehrstufige Kette (`ElektroAuto` → `Auto` → `Fahrzeug`) funktioniert	
`sealed`	`Fahrrad` versiegelt, Verhalten verstanden	
Polymorphie	Menüschleifen kennen nur `Fahrzeug` bzw. Interface-Typen	
Polymorphie	Wiederverwendung der Basisberechnung bei `ElektroAuto`	
Interfaces	Zwei Interfaces, eine Klasse (`Smartphone`) ohne Basisklasse mit beiden	
Interfaces	Interface mit Property und Methode	
Kapselung	Felder `private` , Validierung in Properties/Methoden	
Robustheit	Eingaben geprüft, keine Abstürze	
Transfer	Erweiterung (Etappe 7) ohne Änderung der Auswertungsschleifen	
Reflexion	Fehler-Labor ausgefüllt, Erklär-Check ohne Code beantwortet	

Ampel-Regel: Grün = Michael erklärt es ohne Hilfe an seinem eigenen Code. Gelb = mit Hilfestellung. Rot = betroffene Einzelaufgabe wiederholen und danach die Etappe neu bauen.

---

9. Wenn du hängst

Reihenfolge der Hilfen, bevor du fragst:

1. Zurück zu **deiner Zeichnung**: Stimmt das Bild mit dem Code überein?
2. Den **Compiler-Fehler** Wort für Wort lesen und in eigenen Worten wiedergeben.
3. Im **Debugger** Zeile für Zeile durchgehen und die Variablen beobachten.
4. In der passenden **Einzelaufgabe** und im W3Schools-Kapitel nachlesen (Tabelle in Abschnitt 2).
5. Erst dann Jasmin fragen. Formuliere die Frage so: *“Ich erwarte X, es passiert Y, ich habe Z schon probiert.”*